

ACTIVITY 8 - HACKERRANK ALGORITHMIC PROBLEM-SOLVING & PORTFOLIO INTEGRATION

PORTFOLIO BUILDING B25CS0311 · 3rd Semester B.Tech (Computer Science & Engineering)

Student	Deshna Saanvi M
Program	B.Tech Computer Science & Engineering, 3rd Semester
HackerRank Profile	https://www.hackerrank.com/profile/deshnasaanvii
GitHub Repository	https://github.com/deshnasaanvii-pixel/HackerRank-3rdSem-Portfolio

1. Activity Overview & Objectives

This activity focused on solving five mandatory HackerRank problems, analyzing their time and space complexity, organizing clean C++ solutions in a public GitHub repository, and documenting HackerRank submission and badge evidence. The work emphasizes arrays, strings, dynamic arrays, frequency mapping, and algorithmic efficiency.

2. Mandatory Problem Set & Complexity Analysis

#	Problem	Topic / Key Concept	Target
1	Diagonal Difference	2D Arrays / diagonal traversal	$O(N)$ time $O(1)$ space
2	Dynamic Array	Vectors, nested sequences, XOR	$O(N + Q)$ time $O(N)$ space
3	Time Conversion	Strings, AM/PM conversion	$O(1)$ time $O(1)$ space
4	Compare the Triplets	Element-wise comparison & scoring	$O(1)$ time $O(1)$ space
5	Sparse Arrays	Hash maps / frequency mapping	$O(N + Q)$ time $O(N)$ space

3. Portfolio Integration

The GitHub repository contains a dedicated folder for each problem, solution source code, a root README with profile and repository links, a complexity summary, and a screenshots folder containing submission and badge evidence.

01-Diagonal-Difference	03-Time-Conversion	05-Sparse-Arrays
02-Dynamic-Array	04-Compare-the-Triplets	screenshots/

4. Reflective Summary

This activity helped me understand how algorithm choice affects execution time and memory usage. While solving Diagonal Difference, I learned that both diagonal sums can be calculated in one traversal instead of separate passes. Dynamic Array introduced vectors and XOR to select sequences efficiently. Time Conversion strengthened my understanding of string indexing and conditional logic. Compare the Triplets showed how fixed-size comparisons can be implemented without unnecessary data structures. Sparse Arrays demonstrated the advantage of frequency mapping: storing each string count once avoids repeatedly scanning the complete input list for every query.

A major optimization lesson was to avoid repeated work. When many queries use the same data, preprocessing or frequency counting can reduce computation. I also practiced identifying complexity before writing code, which makes it easier to choose suitable data structures. These problems improved my confidence in writing concise C++ solutions and documenting time and space requirements. Integrating the solutions into GitHub also showed me how coding practice can be organized into a portfolio that can be reviewed by others.

5. Badge & Submission Evidence

Evidence	Repository Evidence
Diagonal Difference	Accepted submission screenshot
Dynamic Array	Accepted submission screenshot
Time Conversion	Accepted submission screenshot
Compare the Triplets	Accepted submission screenshot
Sparse Arrays	Accepted submission screenshot
HackerRank Badge	3-Star badge screenshot

All evidence screenshots are available in the screenshots/ folder of the GitHub repository.

6. Submission Links

HackerRank Profile	https://www.hackerrank.com/profile/deshnasaanvii
GitHub Repository	https://github.com/deshnasaanvii-pixel/HackerRank-3rdSem-Portfolio

Tools Used: HackerRank · C++ · Visual Studio Code · Git · GitHub

Deliverable: Two-page report containing the required profile/repository links, problem complexity analysis, reflective summary, and evidence overview.